

STAT 4710/5710: Homework 4

Ying Jin

Due: November 11, 2025 at 11:59am

Contents

Instructions	1
Case study: predicting breast cancer survival with tree-based methods	3
1 Exploratory data analysis (13 points)	3
1.1 Class distribution (3 points)	3
1.2 Exploring feature distributions (10 points)	4
2 Classification trees (28 points)	7
2.1 Growing a default classification tree (12 points)	7
2.2 Tree pruning and complexity (16 points)	8
2.2.1 Fitting a large tree (10 points)	8
2.2.2 Tree pruning and cross validation (6 points)	10
3 Random forests (30 points)	12
3.1 Running a random forest with default parameters (9 points)	12
3.2 Tuning the random forests (11 points)	13
3.3 Variable importance (10 points)	15
4 Boosting (23 points)	17
4.1 Model tuning (14 points)	17
4.2 Model interpretation (9 points)	20
5 Test set evaluation and comparison (6 points)	22

Instructions

Materials and collaboration

The policy on allowed materials and collaboration is as stated on the Syllabus:

“Students are permitted to work together on homework assignments, but must write up and submit solutions individually. In particular, students may not copy each others’ solutions. Students may consult all course materials, textbooks, the internet, or AI tools (e.g. ChatGPT) to complete their homework. Students may not use solutions to problems that may be available online and/or from past iterations of the course. For each homework, students must disclose all classmates with whom they collaborated, which AI tools they used, and how they used them. Failure to do so will result in a 10-point penalty.”

In accordance with this policy,

Please disclose all classmates with whom you collaborated: NA

Please disclose which AI tools you used, and how you used them: ChatGPT in debugging my r code.

Failure to answer the above questions will result in a 10-point penalty.

Writeup

Use `homework-4-template.Rmd` as a starting point for your writeup, adding your solutions after *****Solution*****. Add your R code using code chunks and add your text answers using ****bold text****. Consult the [preparing reports guide](#) for guidance on compilation, creation of figures and tables, and presentation quality. If the instructions ask you to “print a tibble”, you should print the tibble directly.

Programming

We encourage you to follow the `tidyverse` paradigm for data visualization, manipulation, and wrangling, as in the first homework, to practice tidyverse workflows (e.g., `filter`, `mutate`, `group_by`, `summarize`, `join`, `ggplot2`).

Students who strongly prefer to use Python for homework may do so. If you choose this option, you are **required** to present both code chunks and the corresponding outputs/interpretations in your PDF. In addition, it’s recommended that you:

- Use `pandas` for data manipulation (preferred, since it parallels `dplyr`).
- Use `plotnine` for visualization (preferred, since it parallels `ggplot2`). If using `seaborn`/`matplotlib`, you must still include the required plot elements (labels, 45° line, smoothing curve, facets, etc.).

```
library(tidyverse)
library(cowplot)
library(glmnetUtils)
library(rpart)
library(rpart.plot)
library(kableExtra)
library(randomForest)
library(gbm)
```

Code completion. In some problems, the task is to complete an incomplete code chunk provided. In those problems, in the template, there is `eval = FALSE` in the chunk header to prevent evaluation errors. You should remove `eval = FALSE` after you have completed the code to enable evaluation and printing of results.

Grading

The point value for each problem sub-part is indicated. There are 100 points possible on this homework, evaluated based on the correctness of the numerical results and plots. When a problem asks for a plot:

- **Axes and labels:** Always include informative axis labels and, if appropriate, titles.
- **Required elements:** If the instructions ask for a 45° line, smoothing curve, or facets, these must be included.
- **Clarity:** Points may be deducted if plots are cluttered, unreadable, or missing labels.

Submission

Compile your writeup to PDF and submit to Gradescope.

Case study: predicting breast cancer survival with tree-based methods

```
data = read_csv("cancer.csv")
data = data %>% mutate(overall_survival = factor(overall_survival))
data = data %>%
  mutate(across(where(is.character), as.factor))

# split data
set.seed(4710)
n <- nrow(data)
train_samples <- sample(1:n, size = 0.8 * n, replace = FALSE)
data_train <- data[train_samples,]
data_test <- data[setdiff(1:n, train_samples),]
```

1 Exploratory data analysis (13 points)

1.1 Class distribution (3 points)

1. Solution.

```
# identify distribution of overall survival
table(data_train$overall_survival)
```

```
##
##    0    1
## 485 389
```

```
fraction = 389 / (389 + 485)
print(fraction)
```

```
## [1] 0.445
```

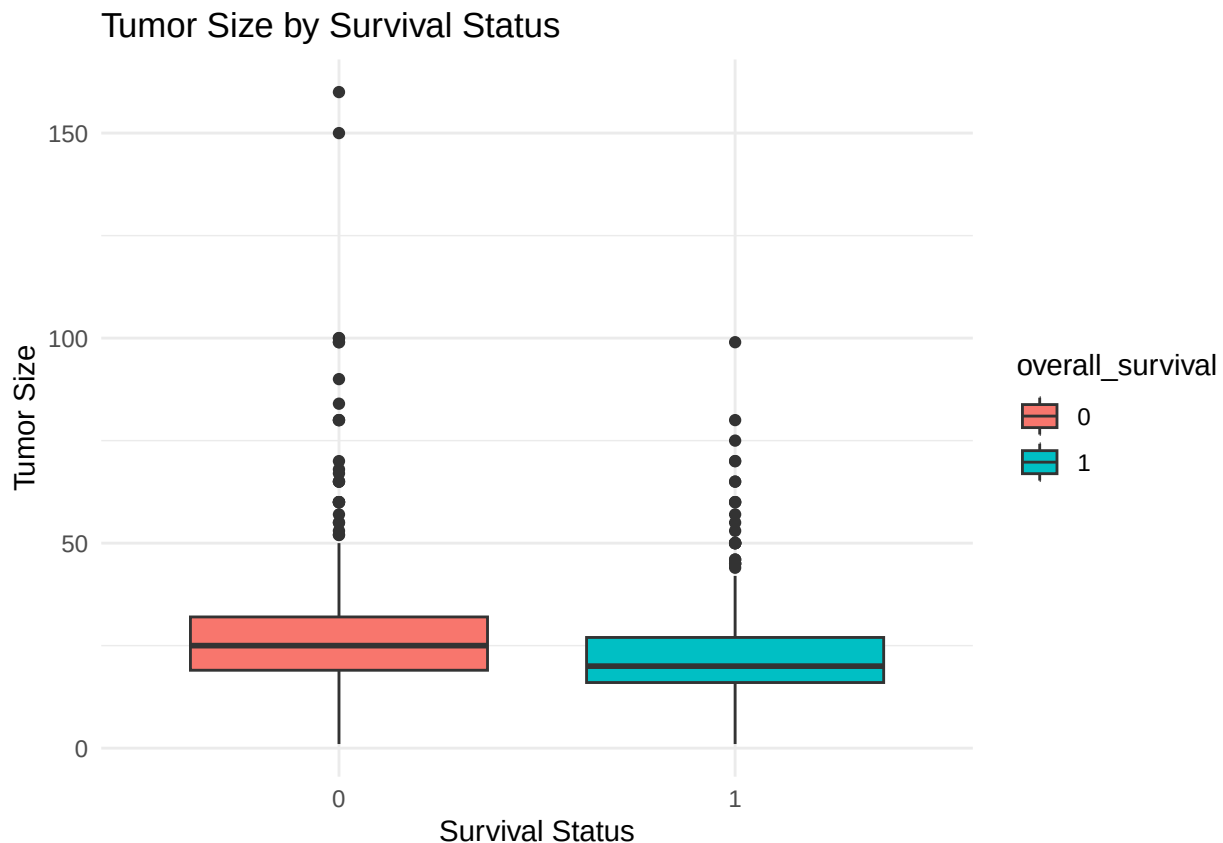
Answer: 44.5 percent of tumors in the training set are survived cases

2. Solution. Answer: assuming the naive classifier will always predicts the majority class, in this case is non-survived, the misclassifications error would be 44.5 percent, which are the estimated proportion of survived cases in testing dataset.

1.2 Exploring feature distributions (10 points)

1. Solution.

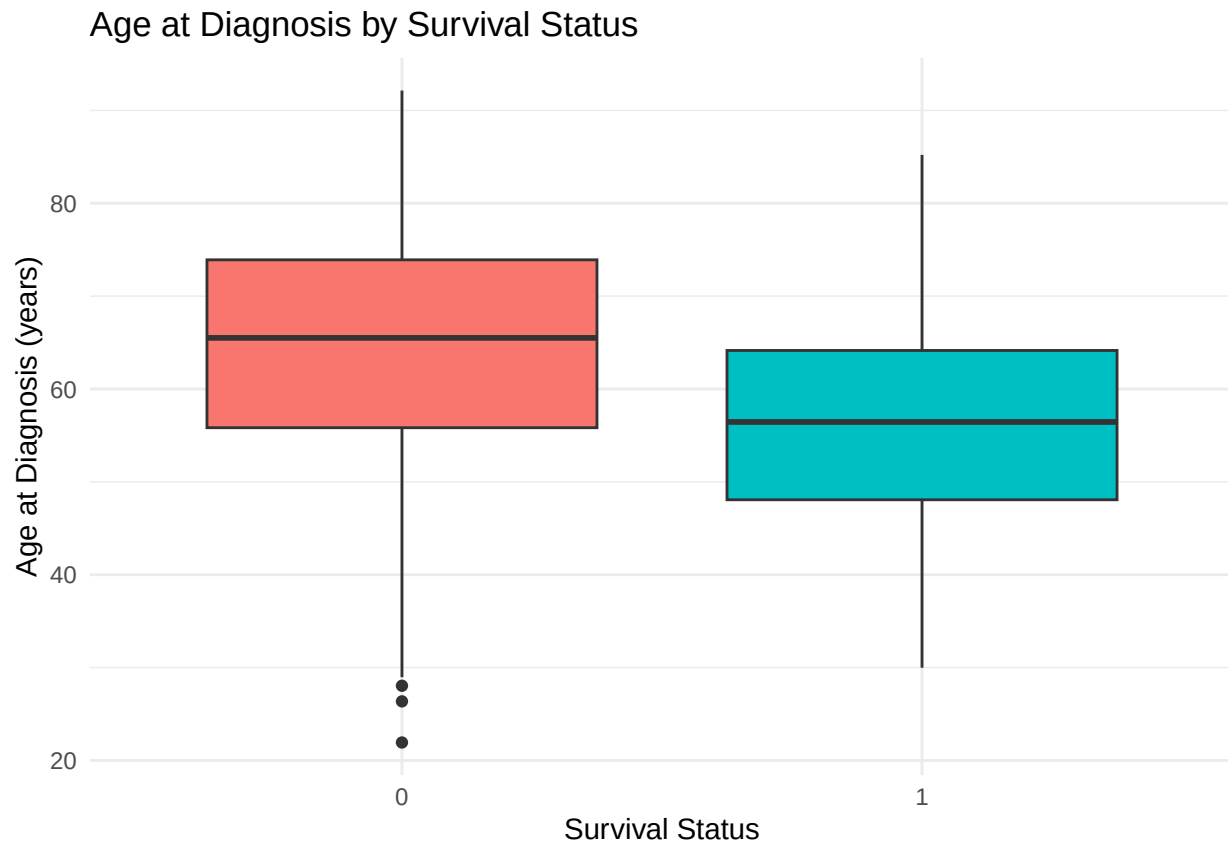
```
# Plot distributions: violin overlay
ggplot(data_train, aes(x = overall_survival, y = tumor_size, fill = overall_survival)) +
  geom_boxplot() +
  labs(title = "Tumor Size by Survival Status",
       x = "Survival Status",
       y = "Tumor Size") +
  theme_minimal()
```



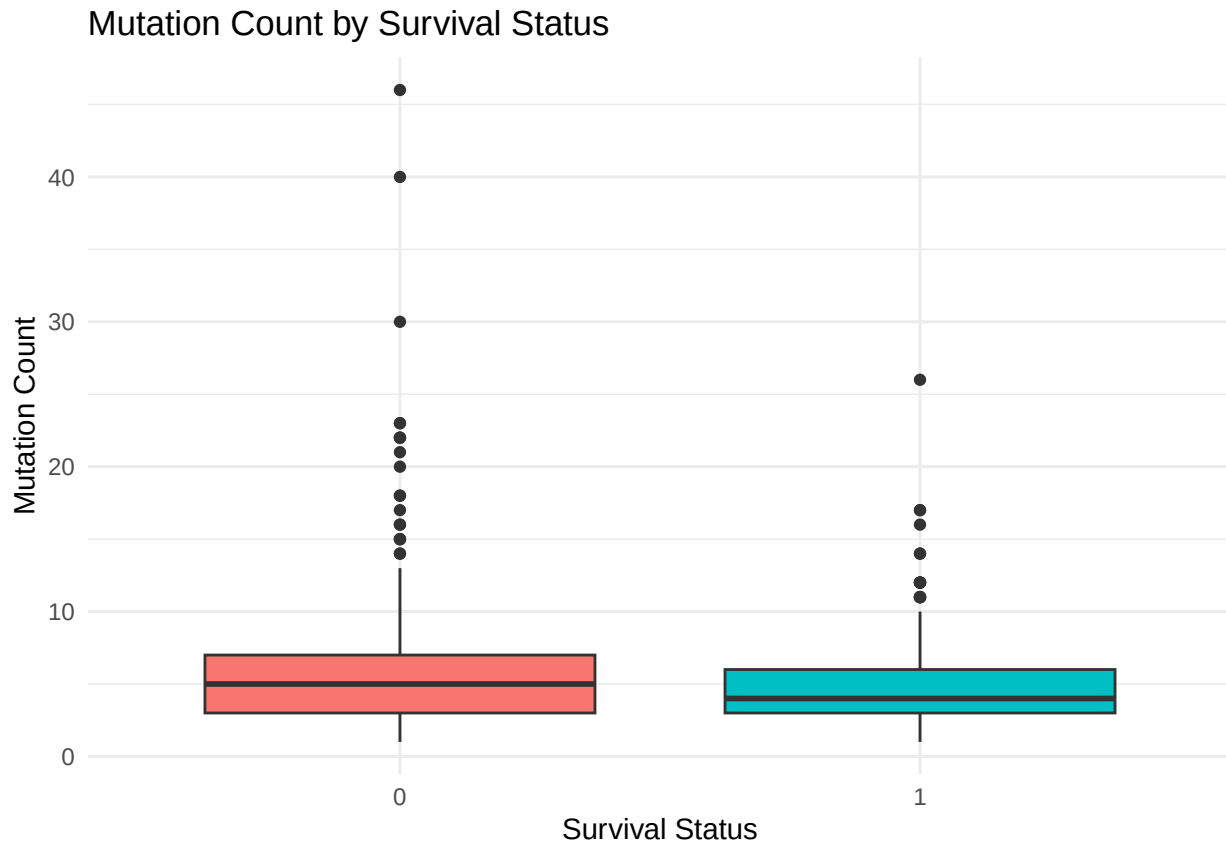
2. Solution.

```
# Boxplot: Age at Diagnosis
ggplot(data_train, aes(x = overall_survival, y = age_at_diagnosis, fill = overall_survival)) +
  geom_boxplot() +
  labs(title = "Age at Diagnosis by Survival Status",
       x = "Survival Status",
```

```
y = "Age at Diagnosis (years)" +  
theme_minimal() +  
theme(legend.position = "none")
```



```
# Boxplot: Mutation Count  
ggplot(data_train, aes(x = overall_survival, y = mutation_count, fill = overall_survival)) +  
  geom_boxplot() +  
  labs(title = "Mutation Count by Survival Status",  
        x = "Survival Status",  
        y = "Mutation Count") +  
  theme_minimal() +  
  theme(legend.position = "none")
```



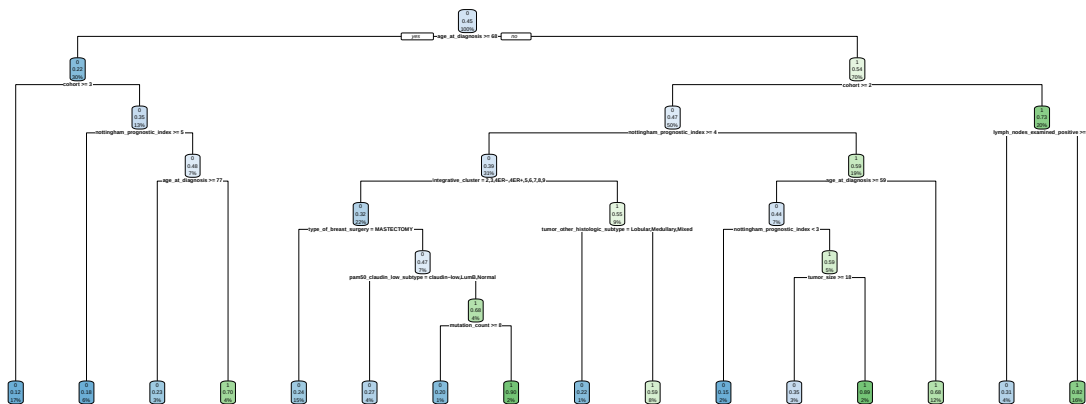
3. **Solution.** Answer: Based on above three plots, we observe the median tumor size, age, and mutation counts of non survived group is larger than survived groups. The distribution for tumor size and mutation counts in both groups are right skewed with non-survived group has few more outliers at the higher end. The distribution for age is roughly symmetric for survived group and slightly left skewed for non survived group.

2 Classification trees (28 points)

2.1 Growing a default classification tree (12 points)

1. Solution.

```
# fit the tree using gini index
tree_fit = rpart(overall_survival ~ .,
  method = "class",
  parms = list(split = "gini"),
  data = data_train)
rpart.plot(tree_fit)
```



2. Solution.

```
# number of terminal nodes
sum(tree_fit$frame$var == "<leaf>")
```

```
## [1] 16
```

```
# number of splits
sum(tree_fit$frame$var != "<leaf>")
```

```
## [1] 15
```

Answer: The tree has 16 terminal nodes and 15 splits in the tree

3. Solution. Answer: the right number on the middle row shows the fraction of survived samples among all the training samples belonging to that node. And the first terminal node counting from left to right has the lowest fraction of survived samples with only 12 percent of survived samples.

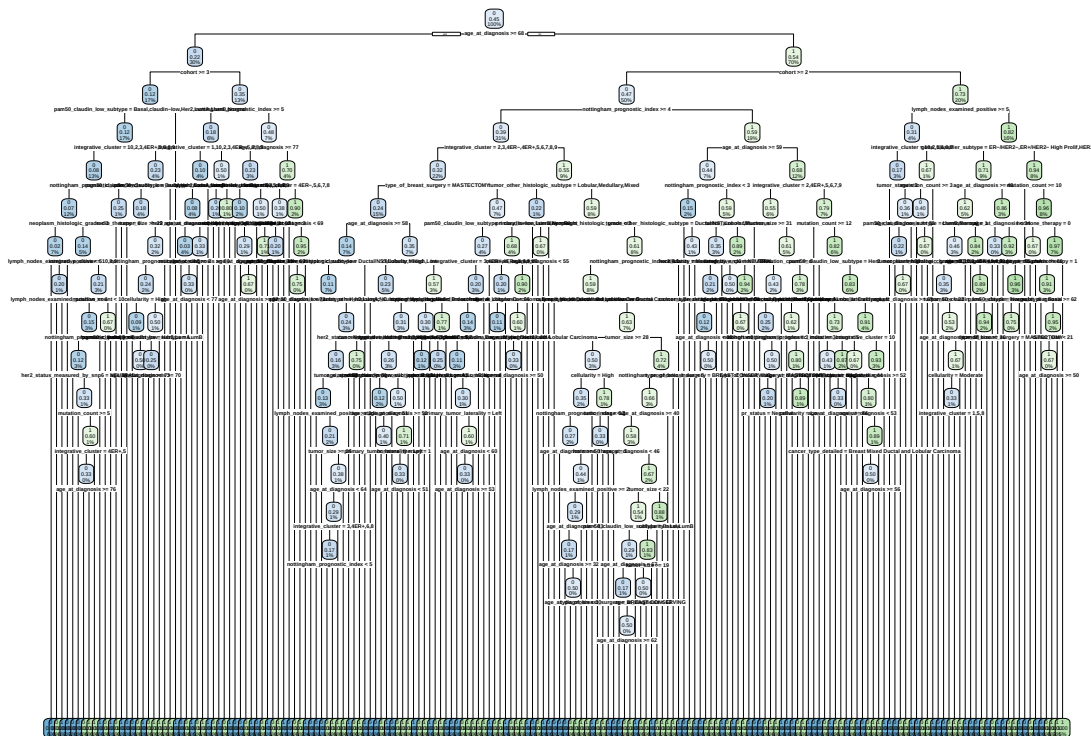
4. Solution. Answer: The lowest fraction of survived sample in the training data happens when age_at_diagnosis is greater or equal to 68, and cohort is greater or equal to 3

2.2 Tree pruning and complexity (16 points)

2.2.1 Fitting a large tree (10 points)

1. Solution. Answer: when the tree is not large enough, the bias will dominate the test error, which makes tree pruning less useful, since it's mainly used to reduce the variance of a large tree.
2. Solution.

```
# fit the tree with controlling function
deep_tree_fit = rpart(overall_survival ~ .,
  method = "class",
  parms = list(split = "gini"),
  control = rpart.control(minsplit= 1,
    minbucket=1,
    cp = 0),
  data = data_train)
rpart.plot(deep_tree_fit)
```



3. Solution.

```
cp_table <- deep_tree_fit$cptable %>%
  as_tibble()
cp_table %>% kable(
  format = "latex", row.names = NA, booktabs = TRUE, digits = 5, col.names = c(
    "CP",
    "Number of splits",
    "Relative error",
```



```

    "CV error",
    "CV std"
  ),
  caption = "This is the CP table for this deepest tree."
) |>
kable_styling(position = "center") |> kable_styling(latex_options = "HOLD_position")

```

Table 1: This is the CP table for this deepest tree.

CP	Number of splits	Relative error	CV error	CV std
0.12596	0	1.0000	1.000	0.0378
0.07841	1	0.8740	0.895	0.0372
0.03085	3	0.7172	0.812	0.0365
0.02057	4	0.6864	0.817	0.0366
0.01799	5	0.6658	0.784	0.0362
0.01414	8	0.6118	0.781	0.0362
0.01285	11	0.5681	0.779	0.0362
0.01114	12	0.5553	0.789	0.0363
0.01028	15	0.5219	0.792	0.0363
0.00771	16	0.5116	0.784	0.0362
0.00643	21	0.4730	0.774	0.0361
0.00600	25	0.4396	0.774	0.0361
0.00514	28	0.4216	0.810	0.0365
0.00450	40	0.3573	0.810	0.0365
0.00386	48	0.3188	0.820	0.0366
0.00321	59	0.2751	0.823	0.0366
0.00257	63	0.2622	0.823	0.0366
0.00193	131	0.0823	0.859	0.0369
0.00171	135	0.0746	0.877	0.0371
0.00154	141	0.0643	0.877	0.0371
0.00129	155	0.0411	0.902	0.0373
0.00000	187	0.0000	0.907	0.0373

```
print(cp_table)
```

```

## # A tibble: 22 x 5
##       CP nsplit 'rel error' xerror  xstd
##   <dbl> <dbl>    <dbl> <dbl> <dbl>
## 1 0.126     0      1      1 0.0378
## 2 0.0784     1    0.874 0.895 0.0372
## 3 0.0308     3    0.717 0.812 0.0365
## 4 0.0206     4    0.686 0.817 0.0366
## 5 0.0180     5    0.666 0.784 0.0362
## 6 0.0141     8    0.612 0.781 0.0362
## 7 0.0129    11    0.568 0.779 0.0362
## 8 0.0111    12    0.555 0.789 0.0363
## 9 0.0103    15    0.522 0.792 0.0363
##10 0.00771   16    0.512 0.784 0.0362
## # i 12 more rows

```

4. Solution.

```
# check the number of leaves of deep tree
sum(deep_tree_fit$frame$var == "<leaf>")
```

```
## [1] 188
```

```
# compute the average obs per node
nrow(data_train) / sum(deep_tree_fit$frame$var == "<leaf>")
```

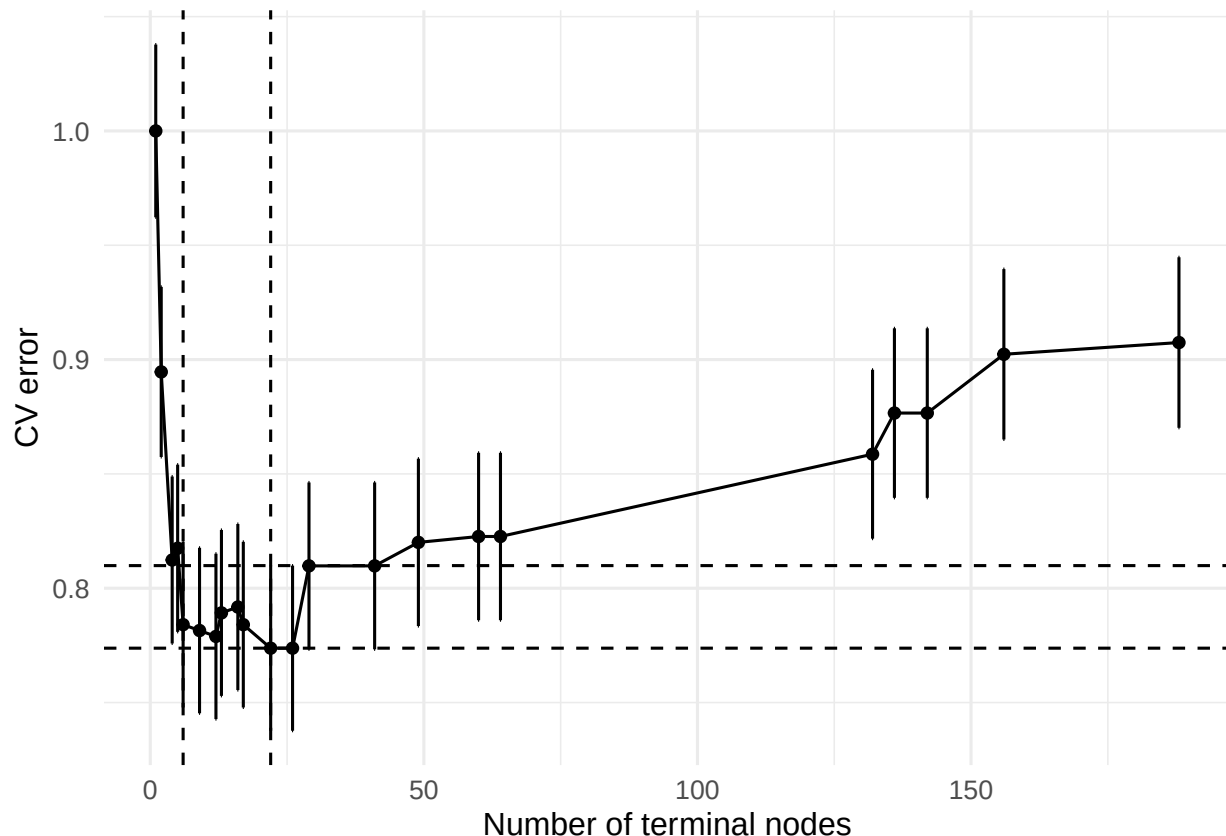
```
## [1] 4.65
```

Answer: Based on the cp table, we have 22 distinct trees in part 3. The biggest tree has 187 splits and 188 terminal nodes. It has 4.65 observations per terminal node on average. It does not equal to 1 because there are multiple observations having similar feature values and are in the same survival class, resulting no further split necessary to reduce Gini impurity.

2.2.2 Tree pruning and cross validation (6 points)

1. Solution.

```
# fit the cv tree function
out = cv_tree(deep_tree_fit)
# generate the cv plot
out$cv_plot
```



2. Solution.

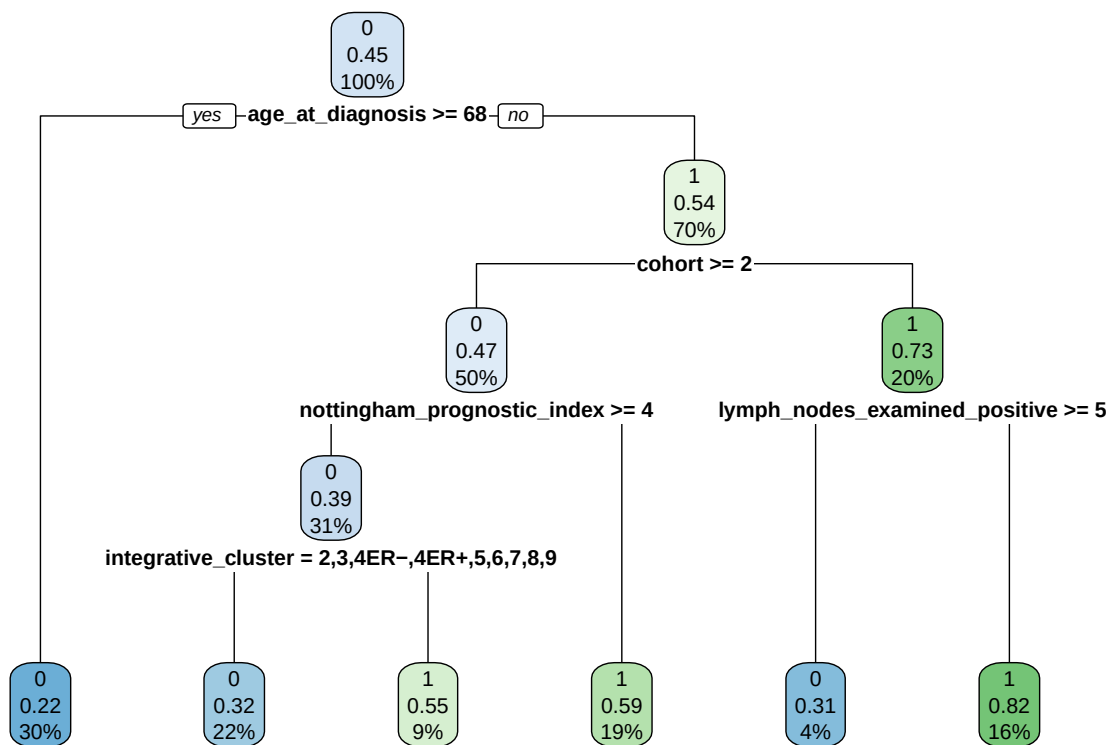
```
# identify the 1se tree terminal nodes
out$nlseaves.1se
```

```
## [1] 6
```

Answer: the 1se tree has 6 terminal nodes, which is less than the default tree in section 2.1, that has 16 terminal nodes.

3. Solution.

```
# prune the optimal tree
optimal_tree = prune(deep_tree_fit, cp = out$CP.1se)
rpart.plot(optimal_tree)
```



3 Random forests (30 points)

3.1 Running a random forest with default parameters (9 points)

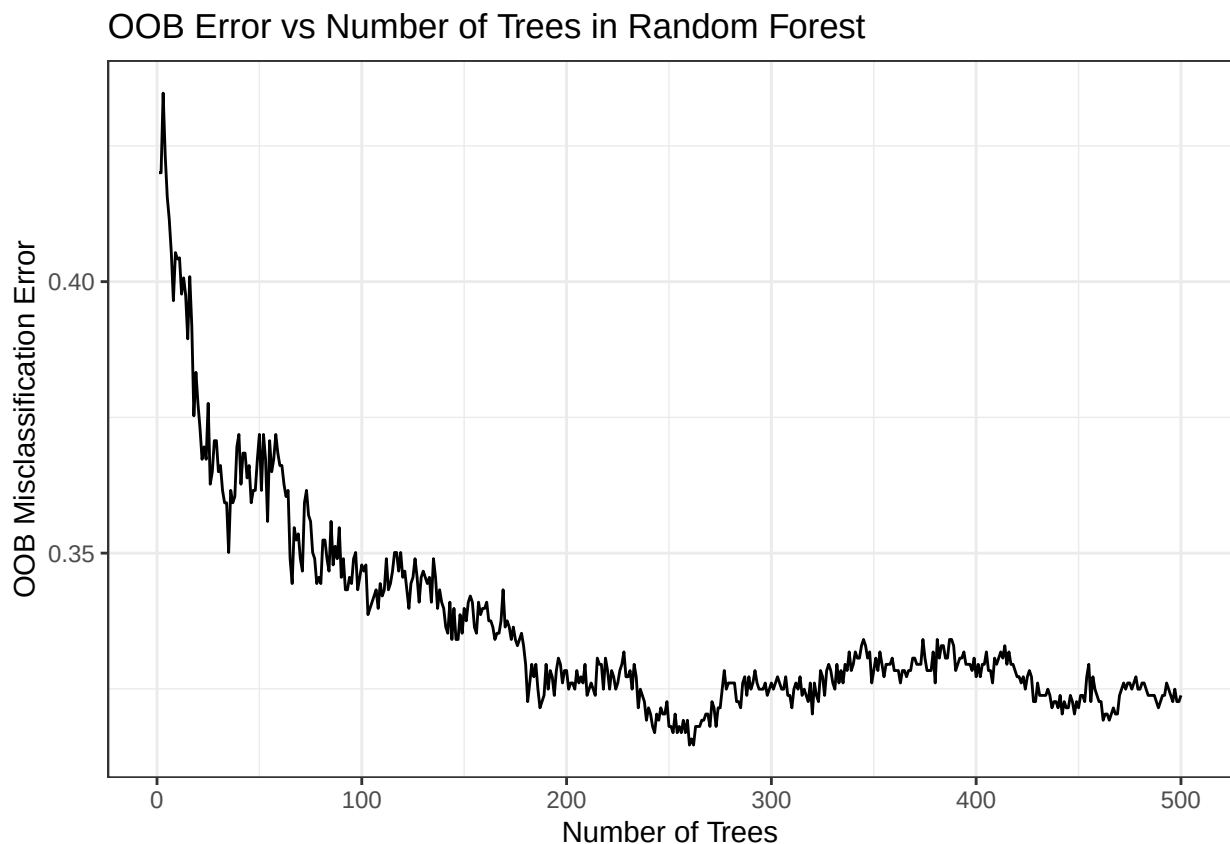
1. Solution.

```
set.seed(4710)
# train the model
rf_fit = randomForest(overall_survival~., data = data_train)
```

2. Solution.

```
# extract the OOB error across trees
oob_data = tibble(
  trees = 1:nrow(rf_fit$err.rate),
  oob_err = rf_fit$err.rate[, "OOB"]
)

# plot
ggplot(oob_data, aes(x = trees, y = oob_err)) +
  geom_line() +
  labs(
    x = "Number of Trees",
    y = "OOB Misclassification Error",
    title = "OOB Error vs Number of Trees in Random Forest"
  )
```



Solution.

3.

```
# identify OOB error for 200 trees
oob_data[200,]
```

```
## # A tibble: 1 x 2
##   trees oob_err
##   <int>   <dbl>
## 1    200   0.328
```

Answer: The OOB estimate for 200 trees is 0.328

4. Solution. Answer: a reasonable number of trees based on OOB error plot is 200 trees.

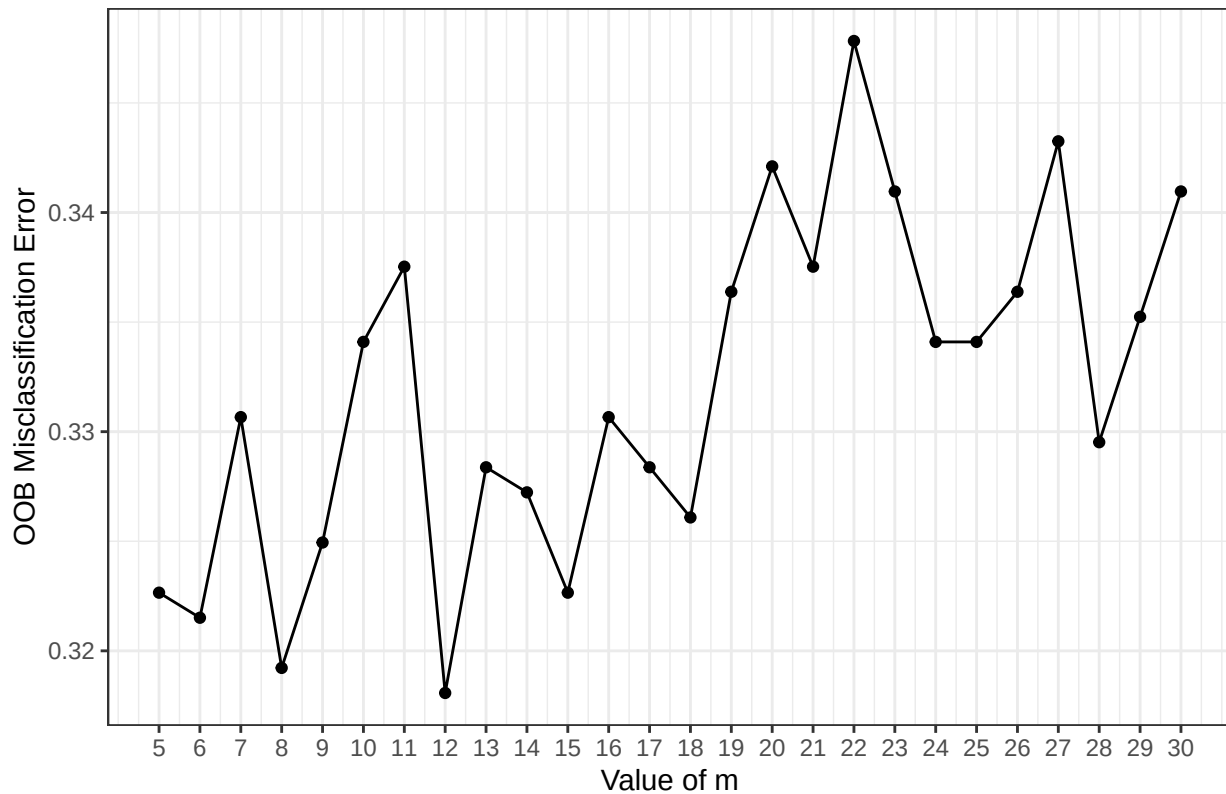
3.2 Tuning the random forests (11 points)

1. Solution.

```
# set seed for reproducibility
set.seed(471)
mvalues <- seq(5, 30)
oob_errors <- numeric(length(mvalues))
ntree <- 500
for (idx in 1:length(mvalues)) {
  m <- mvalues[idx]
  rf_fit <- randomForest(overall_survival ~ ., mtry = m, data = data_train)
  oob_errors[idx] <- rf_fit$err.rate[500, 'OOB']
}

results = tibble(m = mvalues, oob_err = oob_errors)
results |>
  ggplot(aes(x = m, y = oob_err)) +
  geom_line() +
  geom_point() +
  scale_x_continuous(breaks = mvalues) +
  labs(
    x = "Value of m",
    y = "OOB Misclassification Error",
    title = "OOB Error vs M tries in Random Forest"
  )
```

OOB Error vs M tries in Random Forest



2.

Solution.

```
# identify the value min OOB error when m = 12
results |>
  filter(oob_err == min(oob_err))
```

```
## # A tibble: 1 x 2
##       m oob_err
##   <int>   <dbl>
## 1     12  0.318
```

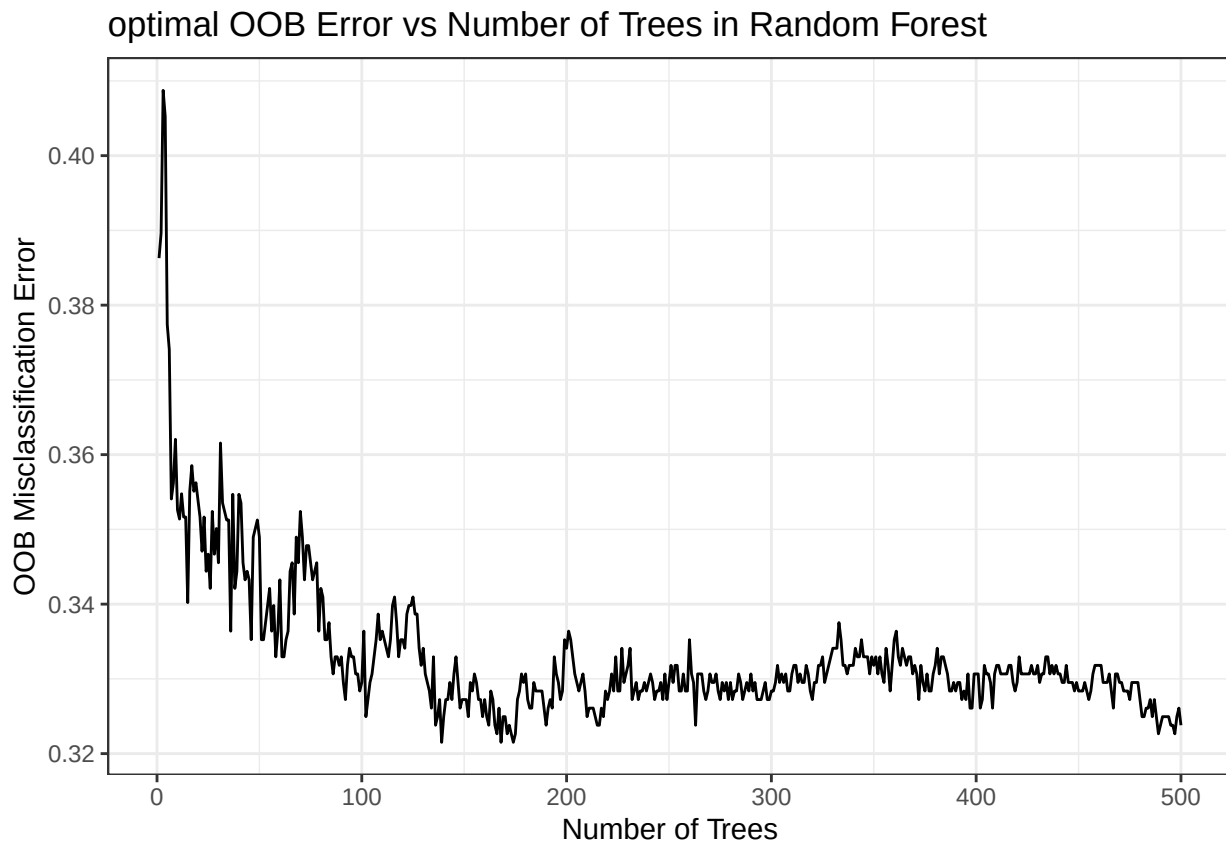
Answer: when $m = 12$ minimizes the OOB error, which would be 0.318

3. Solution.

```
# refit the model with optimal m = 12
set.seed(471)
optimal_rf_fit <- randomForest(overall_survival ~ ., importance=TRUE,
                              mtry = 12, data = data_train)
# extract the OOB error across trees
optimal_oob_data = tibble(
  trees = 1:nrow(optimal_rf_fit$err.rate),
  oob_err = optimal_rf_fit$err.rate[, "OOB"]
)

# plot
```

```
ggplot(optimal_oob_data, aes(x = trees, y = oob_err)) +
  geom_line() +
  labs(
    x = "Number of Trees",
    y = "OOB Misclassification Error",
    title = "optimal OOB Error vs Number of Trees in Random Forest"
  )
```



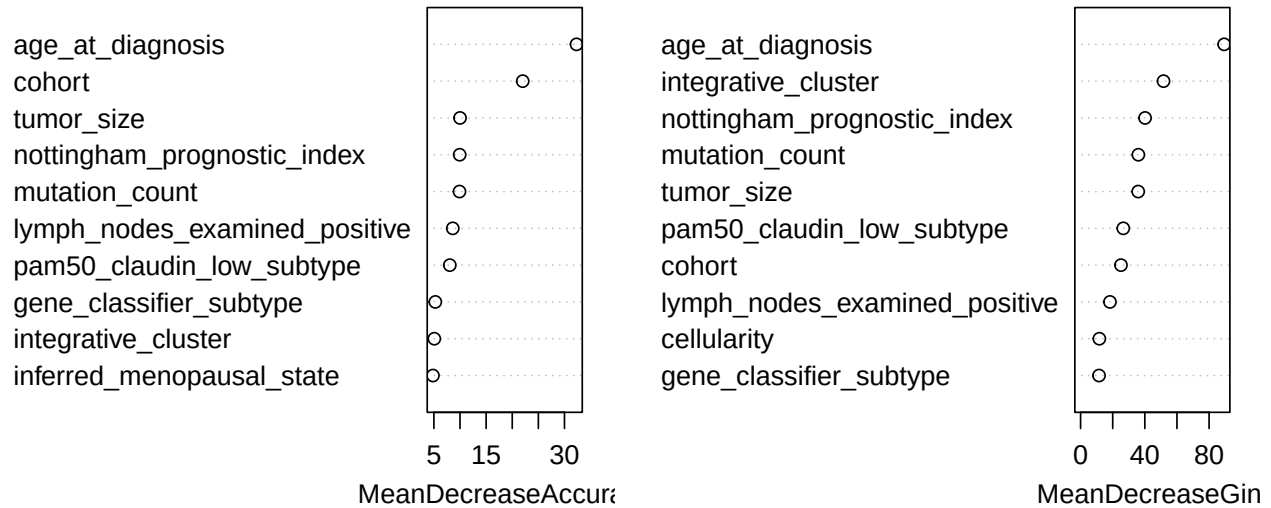
Answer: The error has flattened out

3.3 Variable importance (10 points)

1. Solution.

```
# plot the importance
varImpPlot(optimal_rf_fit, n.var = 10)
```

optimal_rf_fit



2. Solution. Answer: the top 3 important features for accuracy are age_at_diagnosis, cohort and tumor_size; the top 3 important features for Gini index are age_at_diagnosis, integrative_cluster, and nottingham_prognostic_index.

3. Solution. Answer: based on results in part 2, we find it makes sense for age_at_diagnosis to be an important predictor for breast cancer survival, since we would expect younger people are more likely to survive due to better physical conditions.

4 Boosting (23 points)

4.1 Model tuning (14 points)

```
data_train = data_train %>% mutate(overall_survival = 1 * (overall_survival == 1))
data_test = data_test %>% mutate(overall_survival = 1 * (overall_survival == 1))
```

1. Solution.

```
# fit three boosted tree with different interaction depths
set.seed(471)
gbm_fit_1 <- gbm(overall_survival ~ .,
  distribution = "bernoulli",
  n.trees = 1000,
  interaction.depth = 1,
  shrinkage = 0.1,
  cv.folds = 10,
  n.cores = 1,
  verbose = F,
  data = data_train
)
```

```
## CV: 1
## CV: 2
## CV: 3
## CV: 4
## CV: 5
## CV: 6
## CV: 7
## CV: 8
## CV: 9
## CV: 10
```

```
gbm_fit_2 <- gbm(overall_survival ~ .,
  distribution = "bernoulli",
  n.trees = 1000,
  interaction.depth = 2,
  shrinkage = 0.1,
  cv.folds = 10,
  n.cores = 1,
  verbose = F,
  data = data_train
)
```

```
## CV: 1
## CV: 2
## CV: 3
## CV: 4
## CV: 5
## CV: 6
## CV: 7
```

```
## CV: 8
## CV: 9
## CV: 10
```

```
gbm_fit_3 <- gbm(overall_survival ~ .,
  distribution = "bernoulli",
  n.trees = 1000,
  interaction.depth = 3,
  shrinkage = 0.1,
  cv.folds = 10,
  n.cores = 1,
  verbose = F,
  data = data_train
)
```

```
## CV: 1
## CV: 2
## CV: 3
## CV: 4
## CV: 5
## CV: 6
## CV: 7
## CV: 8
## CV: 9
## CV: 10
```

2. Solution.

```
# concat df for 3 interaction depth
gbm_cv_results = bind_rows(
  tibble(
    interaction_depth = 1,
    ntrees = 1:length(gbm_fit_1$cv.error),
    cv_error = gbm_fit_1$cv.error
  ),
  tibble(
    interaction_depth = 2,
    ntrees = 1:length(gbm_fit_2$cv.error),
    cv_error = gbm_fit_2$cv.error
  ),
  tibble(
    interaction_depth = 3,
    ntrees = 1:length(gbm_fit_3$cv.error),
    cv_error = gbm_fit_3$cv.error
  )
)
```

3. Solution.

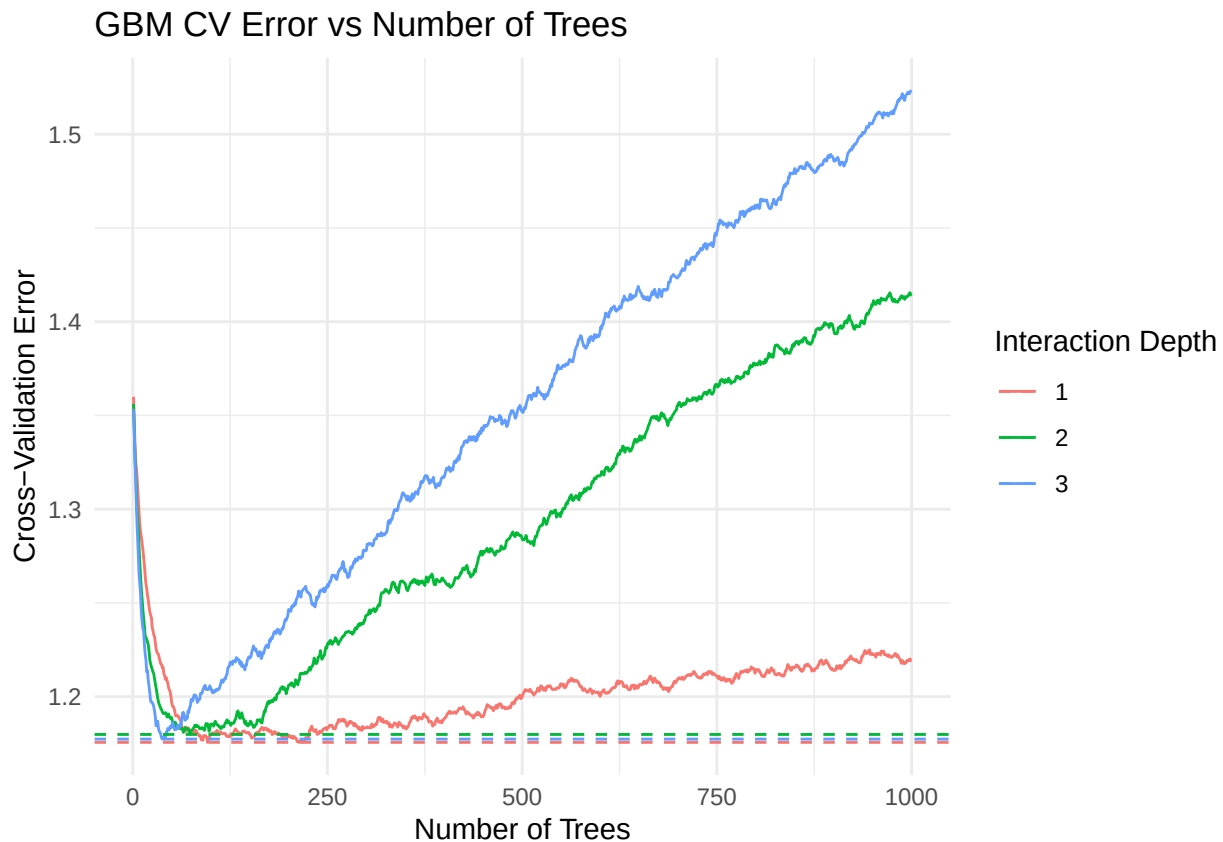
```
# calculate the min error for each interaction depth
min_lines <- gbm_cv_results |>
  group_by(interaction_depth) |>
```

```

summarize(min_cv = min(cv_error))

# create plot for CV errors
ggplot(gbm_cv_results, aes(x = ntrees, y = cv_error, color = factor(interaction_depth))) +
  geom_line() +
  geom_hline(data = min_lines,
             aes(yintercept = min_cv, color = factor(interaction_depth)),
             linetype = "dashed") +
  labs(
    x = "Number of Trees",
    y = "Cross-Validation Error",
    color = "Interaction Depth",
    title = "GBM CV Error vs Number of Trees"
  ) +
  theme_minimal()

```



Solution. Answer: Based on above plots, we observe an initial increase on the number of trees will decrease the CV error, since it reduces the bias; however, when the number of trees keeps increasing, it increases the complexity of the model, hence the variance of the model, particularly for interaction depth being 2 and 3, resulting in increase in CV error. Based on the min CV error in our training data, we identify the best interaction depth is 1.

5. **Solution.**

```

# refit the optimal gb model
gbm_fit_optimal <- gbm_fit_1

```

```
optimal_num_trees <- gbm.perf(gbm_fit_optimal, plot.it = FALSE)
optimal_num_trees
```

```
## [1] 96
```

Answer: the optimal number of trees for our optimal interaction depth (1 in this case) is 96

4.2 Model interpretation (9 points)

1. Solution.

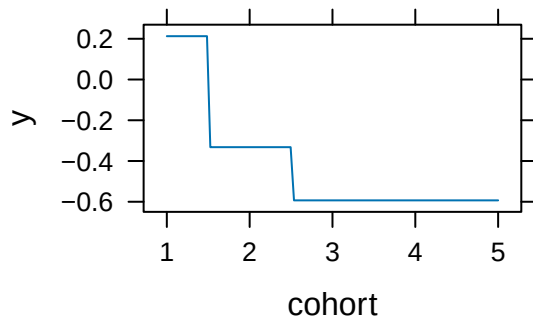
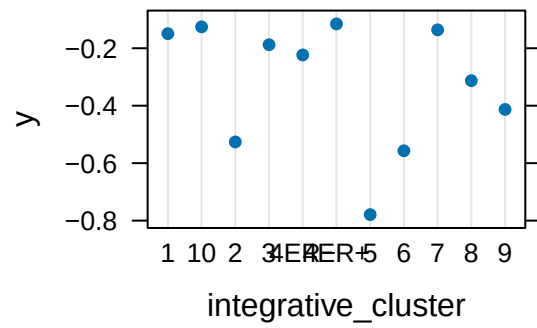
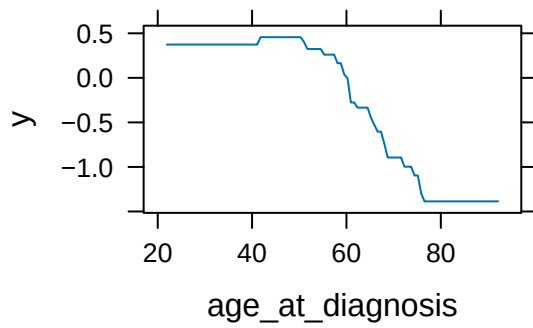
```
print(
  head(
    summary(gbm_fit_optimal, n.trees = optimal_num_trees, plotit = FALSE),
    n = 10)
)
```

```
##                                var rel.inf
## age_at_diagnosis              age_at_diagnosis  34.71
## cohort                        cohort           12.02
## integrative_cluster           integrative_cluster 10.00
## tumor_size                    tumor_size        9.55
## mutation_count                mutation_count     7.11
## lymph_nodes_examined_positive lymph_nodes_examined_positive 6.34
## pam50_claudin_low_subtype      pam50_claudin_low_subtype 4.82
## nottingham_prognostic_index    nottingham_prognostic_index 3.22
## tumor_other_histologic_subtype tumor_other_histologic_subtype 2.82
## type_of_breast_surgery         type_of_breast_surgery 2.68
```

Answer: The top three features are age_at_diagnosis, integrative_cluster, and cohort. It aligns quite well with the top three features in the random forest model evaluated with accuracy, except the random forest include tumor_size as third important feature but gradient boost model include integrative_cluster as third important feature

2. Solution.

```
p1 = plot(gbm_fit_optimal, i.var = "age_at_diagnosis", n.trees = optimal_num_trees)
p2 = plot(gbm_fit_optimal, i.var = "integrative_cluster", n.trees = optimal_num_trees)
p3 = plot(gbm_fit_optimal, i.var = "cohort", n.trees = optimal_num_trees)
plot_grid(p1, p2, p3, align = "h")
```



Answer: the partial response plot for top three features in our gradient boost model makes sense because the probability of survive from breast cancer decrease when someone is older and has more integrative cluster and cohort.

5 Test set evaluation and comparison (6 points)

1. Solution.

```
# identify the ground truth
truth <- data_test$overall_survival
truth01 <- if (is.factor(truth)) as.numeric(truth) else as.numeric(truth)

# get gb model predictions
gbm_probabilities <- predict(gbm_fit_optimal,
  n.trees = optimal_num_trees,
  type = "response", newdata = data_test
)
gbm_pred01 <- as.numeric(gbm_probabilities > 0.5)

# get rf model predictions
rf_predictions <- predict(optimal_rf_fit, newdata = data_test, type = "response")
rf_pred01 = as.numeric(as.character(rf_predictions))

# get tuned decision tree predictions
dt_predictions = predict(optimal_tree, newdata = data_test, type = "class")
dt_pred01 = as.numeric(as.character(dt_predictions))

misclas_err <- tibble(
  model = c("GBM", "Random Forest", "Decision Tree"),
  error = c(
    mean(gbm_pred01 != truth01),
    mean(rf_pred01 != truth01),
    mean(dt_pred01 != truth01)
  )
)

print(misclas_err)
```

```
## # A tibble: 3 x 2
##   model      error
##   <chr>      <dbl>
## 1 GBM        0.297
## 2 Random Forest 0.283
## 3 Decision Tree 0.288
```

Answer: based on the misclassification errors for test data, the random forest model performs the best

2. Solution.

```
# define tree test error
tree_test_error <- mean(dt_pred01 != truth01)

rf_test_err <- apply( t(apply(
  predict(optimal_rf_fit,
    newdata = data_test,
```

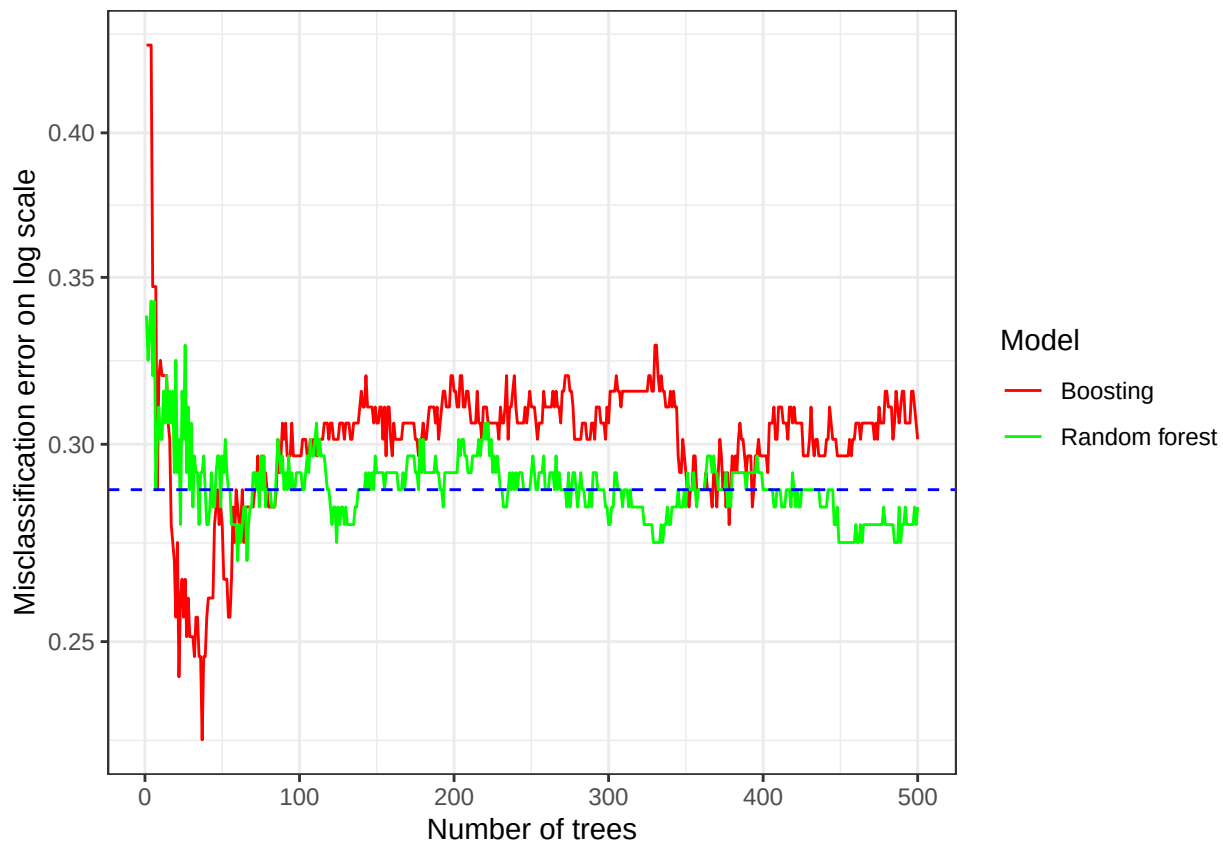
```

      type = "response",
      predict.all = TRUE)$individual,
1, function(row) (as.numeric(cummean(as.numeric(row)) > 0.5))),
2, function(pred) (mean(pred != data_test$overall_survival)))

gbm_test_err <- apply( predict(gbm_fit_optimal, newdata = data_test,
      type = "response", n.trees = 1:500 ), 2,
      function(p) (mean(as.numeric(p > 0.5) != data_test$overall_survival)) )

rf_test_err_tibble <- tibble(ntree = 1:500, error = rf_test_err, method = 'Random forest')
gbm_test_err_tibble <- tibble(ntree = 1:500, error = gbm_test_err, method = 'Boosting')
test_errors <- bind_rows(rf_test_err_tibble, gbm_test_err_tibble)
test_errors |> ggplot(aes(x = ntree, y = error, group = method)) +
  geom_line(aes(color = method)) +
  geom_hline(yintercept = tree_test_error,
    linetype = "dashed", color = "blue") +
  scale_y_log10() +
  scale_color_manual(
    values = c("red", "green") )+
  labs(x = "Number of trees",
    y = "Misclassification error on log scale", color = "Model") +
  theme_bw()

```



Answer: it seems when the number of trees is relatively small, boosting model performs the best and when the number of trees increases, the random forest performs better, since more trees in boosting model will increase model complexity but will not affect complexity in random forest.